

TODO App

Objective

This exercise is intended to evaluate your understanding of full-stack development using your preferred stack.

You'll build a simple TODO app with basic task management functionality, covering both frontend and backend.

Tech Stack Requirements

- **Frontend:** React.js
- **Backend:** Node.js with Express.js is preferred. But you may choose your preferred backend technology.
- **Database:** MongoDB or any other Database technologies

You are free to use any UI libraries or tools that improve your productivity. AI code assistance is acceptable for tedious tasks, however generating large amounts of code using AI may disqualify the submission.

Features

The app should allow users to:

- View TODOs: Display a list of all TODO items.
- Create a TODO: Add a new TODO with a title and optional description.
- Edit a TODO: Update the title and/or description.
- Mark as Done: Toggle a TODO's done status.
- Delete a TODO: Remove a TODO item from the list.

Recommended Directory Structure

You're free to organize your code as you see fit, but here's a suggested structure

```
hiring-fullstack-todo/
├── client/           # React frontend
│   ├── README.md
│   └── ...
├── server/          # Express backend
│   ├── README.md
│   └── ...
└── README.md
```

API Requirements

Your Express backend should expose the following RESTful API endpoints:

Method	Endpoint	Description
GET	/api/todos	Get all TODO items
POST	/api/todos	Create a new TODO item
PUT	/api/todos/:id	Update a TODO (title/description)
PATCH	/api/todos/:id/done	Toggle the done status
DELETE	/api/todos/:id	Delete a TODO

Database Model Example

```
{
  "_id": "string",
  "title": "string",
  "description": "string (optional)",
  "done": "boolean",
  "createdAt": "Timestamp",
  "updatedAt": "Timestamp"
}
```

You are encouraged to use Mongoose for schema modeling.

Frontend Expectations

- Display all TODOs in a clean and simple UI
- Provide a form to add new TODOs
- Allow editing a TODO (title/description)
- Provide a way to mark as done/undone (e.g., checkbox, button)
- Add a delete button
- Clearly show completed tasks (e.g., strikethrough or faded style)

Functional Expectations

- The frontend should talk to the backend via HTTP API
- The backend should persist TODOs in MongoDB
- Handle loading and error states gracefully

Bonus Points For:

- Well written README files
- Form validation
- User-friendly error messages
- Optimistic UI updates
- Nice touches in UX (e.g., animations, transitions)
- Monorepo setup using an appropriate technology

Submission Instructions

Please follow these steps for submission:

- Create a public repository from your own GitHub account.
- Create a new branch named develop in your fork.
- Complete the assignment on the develop branch.
- Create a Pull Request to your own fork (develop → main).
- Fill out the PR body, including:
 - Summary of what you built
 - Solution rationale and explanations for choices you made
 - A short demo video of the platform
 - A 5 minute walkthrough of the code, explaining the architecture/code.
- Fill the submission form given in the email.

Submissions may not be considered if instructions are not followed properly

Time Expectation

This task is expected to take 6-8 hours. Please don't worry about making it perfect, we're primarily looking at how you approach full-stack development and structure your solution.

Your submission must include:

- README.md file for the frontend with:
 - instructions on how to set up and run the app
 - Any assumptions or limitations
- README.md file for the backend with:
 - How to set up and run the backend
 - MongoDB connection notes (e.g., Atlas or local)
 - Any assumptions or limitations

Thank you for taking the time to complete this assignment! We're looking forward to seeing what you build.